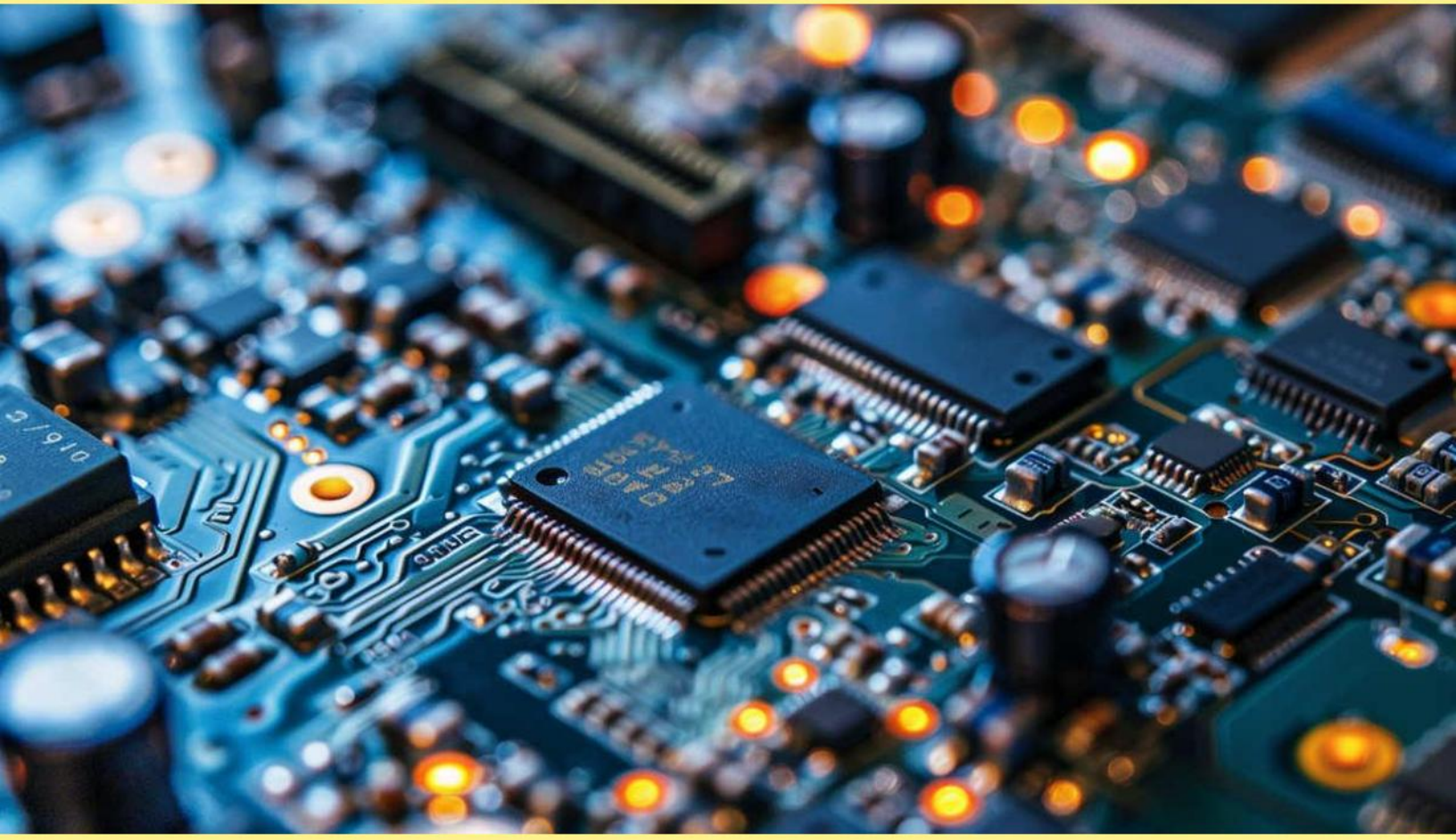
CPI A2 – PROJET SYSTÈMES EMBARQUÉS

PROSIT 6 – INTERRUPTIONS



CESI
ÉCOLE D'INGÉNIEURS

Contexte :

L’équipe récup les données des capteurs, un bouton doit permettre de changer le mode avec un appuis, avec le code les appuis ne sont pas bien détectés

Synchroniser LEDs et boutons

Mots inconnus :

Interruption : signal qui suspend le programme pour traiter un événement urgent.
Scrutation : le programme vérifie lui-même l’état d’une entrée, en boucle.
EXTI : détecte un front sur une broche et déclenche une interruption.
NVIC : gère les interruptions et leurs priorités.
ISR : fonction exécutée lors d’une interruption.
GPIO : broche d’entrée ou de sortie (bouton, LED).
ALU : unité de calcul du processeur.
UART : liaison série asynchrone sur 2 fils (TX/RX).
SPI : bus série rapide, maître/esclave, 4 fils.
I2C : bus série sur 2 fils, esclaves identifiés par une adresse.
ADC : convertit une tension analogique en valeur numérique.

Problématique :

Comment détecter un appui sur un bouton et mesurer sa durée pendant la lecture de données ?

Plan d'action :

- Se documenter sur les mots clés [30 min]
- Analyser le code et comprendre le problème du bouton mal détecté [3h30]
- Résoudre le problème [1 h]

Réalisation :

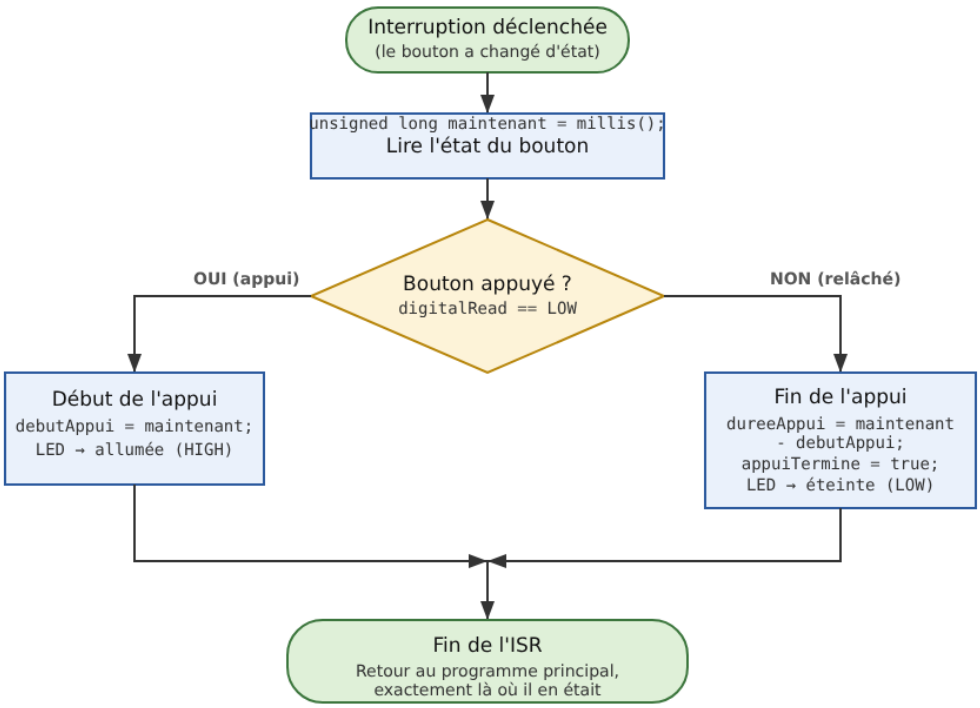
Analyse du problème :

Dans le code fourni, le bouton est lu une seule fois par tour de boucle, juste avant acquisition(). Cette fonction simule le vrai travail de la station (lecture des capteurs, écriture SD) et bloque le programme 3 à 7 secondes (HAL_Delay). Le microcontrôleur n'a qu'un seul cœur : pendant ce délai, il ne regarde plus le bouton du tout. Un appui est donc soit raté, soit pris en compte plusieurs secondes trop tard, ce qui explique la désynchronisation de la LED décrite dans l'énoncé.

Solution retenue :

On utilise une interruption (attachInterrupt) sur la broche du bouton. Le matériel surveille le bouton en continu, même pendant acquisition() : dès que le bouton change d'état, le programme est mis en pause, une petite fonction (l'ISR) s'exécute, puis le programme reprend exactement où il en était. L'ISR note l'heure au début de l'appui, calcule la durée au relâchement (millis()), et allume/éteint la LED immédiatement, sans attendre la fin de l'acquisition.

Organigramme de l'ISR :



Code (Arduino) :

```ino
#include <Arduino.h>
#define LED_PIN LED_BUILTIN
#define BUTTON_PIN 2
#define BUTTON_PRESSED LOW

volatile unsigned long debutAppui = 0;   // instant (ms) ou le bouton a ete appuye
volatile unsigned long dureeAppui = 0;   // duree du dernier appui (ms)
volatile bool appuiTermine = false;      // vrai quand une duree est prete a etre lue

void acquisition();

// routine d'interruption (ISR), appelee automatiquement a chaque
// changement d'etat du bouton, meme pendant acquisition()
void isrBouton() {
  unsigned long maintenant = millis();

  if (digitalRead(BUTTON_PIN) == BUTTON_PRESSED) {   // bouton appuye
    debutAppui = maintenant;                          // on lance le chrono
    digitalWrite(LED_PIN, HIGH);                      // LED allumee
  } else {                                            // bouton relache
    dureeAppui = maintenant - debutAppui;             // duree = fin - debut
    appuiTermine = true;                              // previent loop()
    digitalWrite(LED_PIN, LOW);                       // LED eteinte
  }
}

void setup() {
  Serial.begin(9600);
  pinMode(LED_PIN, OUTPUT);
  pinMode(BUTTON_PIN, INPUT_PULLUP);
  digitalWrite(LED_PIN, LOW);
  attachInterrupt(digitalPinToInterrupt(BUTTON_PIN), isrBouton, CHANGE);
}

void loop() {
  acquisition();

  if (appuiTermine) {
    noInterrupts();
    unsigned long duree = dureeAppui;
    appuiTermine = false;
    interrupts();

    Serial.print("Appui de ");
    Serial.print(duree);
    Serial.println(" ms");
  }
}

void acquisition() {
  unsigned long acquisition_duration = 3000UL + (millis() % 4000UL);
  delay(acquisition_duration);
}
```